



INDIA.RUNS

Build what next India runs on

Team Name : Team Praneeth

Team Leader Name : Praneeth

Problem Statement : Data & AI Challenge – Intelligent Candidate Discovery & Ranking

Solution Overview

- A deterministic, rule-based ranking engine that scores all 100,000 candidates against the Senior AI/ML Engineer JD and outputs the top-100 in ~10 seconds on CPU – pure Python standard library, no network, no GPU, no LLM calls.

What differentiates it from traditional candidate matching:

Encodes the JD's actual hiring rubric as 7 weighted scoring dimensions instead of embedding-cosine keyword similarity.

Values skill QUALITY over quantity: proficiency x months-of-use, so an 'expert with 0 months' scores zero.

Weights hireability, not just fit: platform recency, recruiter response rate, and notice period modify the score.

Built-in impossible-profile (honeypot) detection zeroes out fabricated candidates.

Fully deterministic and reproducible: same input, byte-identical output, every run.

JD Understanding & Candidate Evaluation

- **Key requirements extracted from the JD:**

5-9 years experience (ideal 6-8), with 4-5 in applied ML at product companies; production embeddings/retrieval + vector DB experience; strong Python; ranking evaluation (NDCG, MRR); India, Pune/Noida preferred; sub-30-day notice; active on the Redrob platform. Explicit anti-signals: consulting-only careers, CV/speech-only backgrounds, title-chasers, framework enthusiasts.

Most important candidate signals (in scoring order):

Current + historical job titles (25%) -> a Marketing Manager with AI keywords is not a fit.

Career history text evidence (25%) -> did they actually build ranking / search / recommendation systems?

Skills with proficiency x duration (20%), behavioral engagement (15%), experience band (8%), location (4%), education (3%).

Beyond keyword matching: title tiering, career-description evidence mining, duration-weighted skill trust, and cross-field consistency checks catch what keyword counts cannot.

Ranking Methodology

- $\text{Score} = (0.25 \text{ Title} + 0.25 \text{ Career} + 0.20 \text{ Skills} + 0.15 \text{ Behavioral} + 0.08 \text{ Experience} + 0.04 \text{ Location} + 0.03 \text{ Education}) \times \text{HoneypotMultiplier}$

Title: 5-tier classification (Staff ML / Search / RecSys engineer = 1.0 ... non-technical = 0.0), blending current (70%) and best historical (30%) title.

Career: product-vs-consulting company weighting (all-consulting = 0.25x penalty), industry relevance, JD-keyword evidence in role descriptions, tenure health (job-hopper penalty).

Skills: must-have JD skills weighted by proficiency x duration_months; wrong-domain (CV/speech) and keyword-stuffing penalties; Redrob assessment scores.

Behavioral: last-active recency, recruiter response rate & speed, open-to-work + notice period, market demand, interview/offer reliability.

All 100K candidates are streamed and scored; sorted by (-score, candidate_id) so equal scores tie-break deterministically; top-100 selected with per-candidate reasoning.

No black-box model: every heuristic is a JD requirement made executable - fully explainable and defensible.

Explainability & Data Validation

- **Explainability:**

Every top-100 row carries reasoning assembled ONLY from facts present in that candidate's profile: title, company, years, actual matching skills, education tier, behavioral signals - plus honest concerns (long notice, low response rate, non-India location). Hallucination is prevented by construction: the generator can only reference profile fields, never generate free text. Verified programmatically - 100/100 reasoning strings match the underlying profiles exactly, all unique.

Data validation / suspicious-profile handling (honeypot detector):

Employment starting before the company was founded (checked against real founding years: Krutrim 2023, Sarvam AI 2023, Glance 2019, ...).

Stated years_of_experience exceeding the entire career timeline; summary text contradicting the experience field.

'Expert' skills with 0 months of use; mass all-expert skill lists; non-technical titles with deep ML skill stacks.

Suspicion maps to a 1.0 -> 0.0 multiplier. Final top-100 audit: 0 impossible profiles (disqualification threshold is 10).

End-to-End Workflow

- 1. Stream candidates.jsonl line-by-line (constant memory, no full-file load).
- 2. For each candidate, compute the 7 component scores (title, career, skills, behavioral, experience, location, education).
- 3. Combine via the weighted sum; apply the honeypot multiplier (1.0 clean -> 0.0 impossible).
- 4. Global sort by (-score, candidate_id ascending) - deterministic tie-breaking per the spec.
- 5. Select top-100; round scores to 4 decimals; re-sort on rounded scores to keep the validator's tie-break rule intact.
- 6. Generate fact-grounded reasoning for each of the 100 candidates.
- 7. Write submission.csv (validated by the official validate_submission.py).

Single command: `python rank.py --candidates ./candidates.jsonl --out ./submission.csv`

End-to-end: ~10 seconds for 100,000 candidates on a CPU-only machine.

System Architecture

candidates.jsonl (100K)

| stream, one candidate at a time

v

rank.py orchestrator

-- scoring/title_scorer.py	(25%)	5-tier title classification
-- scoring/career_scorer.py	(25%)	companies, industries, JD-evidence, tenure
-- scoring/skills_scorer.py	(20%)	proficiency x duration, stuffing penalty
-- scoring/behavioral_scorer.py	(15%)	recency, response rate, notice period
-- scoring/experience_scorer.py	(8%)	5-9 yr bell curve
-- scoring/location_scorer.py	(4%)	India / Pune / Noida / relocation
-- scoring/education_scorer.py	(3%)	field, tier, degree
-- scoring/honeypot_detector.py	(x multiplier)	impossible-profile filter
-- scoring/reasoning_generator.py		fact-grounded explanations

v

sort (-score, candidate_id) -> top-100 -> submission.csv

Results & Performance

- **Ranking quality (final top-100):**

100/100 hold ML/AI/search/recommendation titles; 94/100 based in India; 86/100 inside the 5-9 year band.
0 honeypots / impossible profiles (independently re-audited after ranking; disqualification threshold is 10).
0 consulting-only careers; 0 keyword stuffers (lowest avg expert-skill duration in top-100: 55 months).
The only two candidates in the pool matching ALL SEVEN of the JD's ideal-profile criteria rank #3 and #8.
100/100 unique, fact-verified reasoning strings.

Compute constraints (limit vs measured):

Runtime: 5 min allowed -> ~10 s actual. Memory: 16 GB allowed -> ~1.8 GB peak.
CPU-only, zero network calls, zero GPU - verified in a fresh clean-room clone that reproduced the CSV byte-for-byte.
Passes the official `validate_submission.py` with no issues.

Technologies Used

- Python 3.11+ standard library only - json, csv, datetime, re, argparse, pathlib.

Why this stack:

Determinism & reproducibility: no model weights, no random seeds, no API drift. Stage-3 reproduction cannot fail.

Speed: streaming + pure-Python heuristics rank 100K candidates in ~10 s - 30x under the 5-minute budget.

Zero dependencies: requirements.txt is intentionally empty; nothing to install, nothing to break in the sandbox.

Explainability: every scoring rule maps 1:1 to a sentence in the JD - defensible line-by-line at interview.

Deliberately avoided: per-candidate LLM calls (cannot fit the compute budget and hallucinate), embedding similarity as the primary signal (falls for keyword stuffers - the exact trap the dataset sets).

AI tools declared: Claude (architecture discussion, scoring design, implementation & review). No candidate data was sent to any LLM.

Submission Assets

GitHub repository (public):

<https://github.com/gpr-27/redrob-candidate-ranking>

Live Sandbox Demo (verified):

<https://huggingface.co/spaces/praneethg27/redrob-ranker>

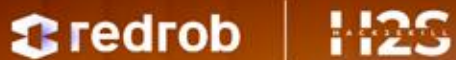
Reproduce command (from repo README):

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

Repo contents: full source (rank.py + 9 scoring modules), README with methodology and setup, requirements.txt, submission_metadata.yaml, submission.csv, official validator.

Ranked output: submission.csv / submission.xlsx (top-100 with rank, score, and per-candidate reasoning).

Sandbox accepts custom uploads and produces valid submission.csv.



INDIA.RUNS

Build what next India runs on

THANK YOU